Professional Self-Assessment

Jose Fabian Then | CS 499: Computer Science Capstone

Completing the Computer Science program and building this ePortfolio has given me something I did not have when I started: a concrete, evidence-backed way to show what I can do, rather than just claim it. I am transitioning into software engineering after more than fourteen years as a paraprofessional and technology lead in the New York City public school system, and alongside that I have worked as a freelance web developer since 2018. The degree gave that experience a formal foundation, and this capstone turned that foundation into demonstrable proof. Over the course of this program I moved from thinking of myself as someone who could get software to work to someone who can critically evaluate, improve, secure, and clearly explain software, which is the difference between writing code and practicing software engineering. This self-assessment introduces who I am as a computer science professional and sets up the three enhanced artifacts that follow, each of which backs up these claims with real, working code.

Collaboration in a Team Environment

Much of my collaboration experience comes from outside the artifacts in this portfolio. As a technology lead in a NYC public school, I spent years working inside a team of educators and administrators where almost no one shared my technical background, which taught me how to contribute to shared goals without assuming everyone speaks my language. The Computer Science program reinforced that skill in a more technical setting. In CS 250, Software Development Lifecycle, I worked within Agile and Scrum frameworks, learning how user stories, sprints, and iterative delivery keep a team aligned, and in CS 320, Software Testing, I learned how a shared testing and quality discipline lets multiple developers trust one another's code. The code review I produced for this capstone is itself a collaborative artifact: a code review

exists to open a dialogue with teammates and reviewers, and producing one trained me to present my own work for critique and to frame technical findings for an audience of peers or a manager rather than only for myself.

Communicating with Stakeholders

Communicating technical information to non-technical stakeholders is a skill I have practiced for years, translating technology decisions for school administrators, parents, and staff who needed to understand what a system did and why it mattered without needing to understand how it worked. This program sharpened that into professional-quality communication. The clearest evidence in this portfolio is my code review video, where I had to walk through existing code, explain its weaknesses, and justify a plan of enhancements clearly enough for a non-specialist audience to follow. The written narratives accompanying each artifact do the same in written form, explaining not just what I changed but why each change mattered. Being able to explain a design decision, and the tradeoffs behind it, in plain language is something I consider as important as the decision itself, because software that no one can explain is software no team can maintain.

Data Structures and Algorithms

My work in data structures and algorithms is represented most directly by my Autonomous Navigation AI Agent, a deep Q-learning agent, but the foundation was built across the program in courses like CS 260, Data Structures and Algorithms. What I value most from this area is not just knowing which structures and algorithms exist, but knowing how to choose among them and justify the choice. In enhancing my Q-learning agent, I replaced an inefficient list-based replay buffer that evicted old entries in linear time with a fixed-length deque that does so in constant time, a concrete complexity improvement I can explain in Big-O terms. More importantly, I built

and benchmarked a simpler tabular Q-learning agent against the full neural network, demonstrating with real measurements that the simpler approach converged in a fraction of the time on a problem small enough not to require the more complex one. That ability to recognize when a more complex solution is not justified, and to prove it rather than assert it, is exactly the kind of engineering judgment that becomes more valuable as tools increasingly generate initial code.

Software Engineering and Database

My software engineering and database skills are represented by two artifacts: my Travlr Getaways full-stack booking platform and my Grazioso Salvare animal shelter dashboard. In the software engineering enhancement, I refactored inconsistent asynchronous error handling into a uniform pattern, added real input validation, corrected incorrect HTTP status codes, and removed duplicated logic, the kind of maintainability and correctness work that separates production-quality code from code that merely runs. In the database enhancement, I restored a corrupted data-access module to working order, completed a CRUD contract that had promised operations it never implemented, and added indexing on the fields the application queries most. Beyond the artifacts, I have continued building full-stack projects on my own, including an Interview Coach application and an autonomous AI job-search agent, which keep my skills current with the tools and practices employers actually use. Across all of it, my focus has been on writing software that is maintainable, correct, and understandable by the next person who has to work on it.

Security

Developing a security mindset was one of the most significant shifts in how I approach code during this program, reinforced by taking CS 405, Secure Coding, in parallel with this capstone. A security mindset means anticipating how a system can be misused or can fail, not just how it is

supposed to work. In my software engineering enhancement, I added protection against brute-force login attempts, replaced raw error responses that leaked internal details with sanitized messages, and added a duplicate-account check rather than letting the database surface its own errors to users. In my database enhancement, I URL-encoded credentials so that special characters could not corrupt or expose a connection string, and I made every data operation fail safely rather than crash on a bad connection. These are not dramatic, movie-style security measures; they are the quiet, disciplined habits that actually keep real systems and real user data safe, and they now inform how I write every piece of code.

How the Artifacts Fit Together

The three artifacts in this portfolio were chosen so that, together, they demonstrate competence across the full breadth of computer science rather than depth in only one area. Travlr Getaways represents software design and engineering through a full-stack web application; the Autonomous Navigation AI Agent represents algorithms and data structures through reinforcement learning and a measured design tradeoff; and the Grazioso Salvare dashboard represents databases through a secure, complete data-access layer. Each began as real coursework, and each was enhanced through the same disciplined process: a critical code review to find genuine weaknesses, a specific plan to address them, and a tested implementation of that plan. Presented alongside my professional self-assessment, they are meant to show not only that I can build software, but that I can evaluate it honestly, improve it deliberately, secure it thoughtfully, and explain it clearly. That combination, more than any single technology on my resume, is what I believe makes me ready to contribute as a software engineer. The enhanced artifacts and their detailed narratives follow.